INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Desarrollo de software para juegos				
TÍTULO DE LA PRÁCTICA:	Predicción y Cálculo de Trayectorias de Proyectiles				
NÚMERO DE PRÁCTICA:	05	**AÑO LECTIVO:**	2026B	**NRO. SEMESTRE:**	IX
FECHA DE PRESENTACIÓN:	13/10/2026	**HORA DE PRESENTACIÓN:**	07:00 am		
INTEGRANTE (s): **Perez Huamani Jeremy Joshua**				**NOTA:**	
DOCENTE(s): **Jose Alfredo Sulla Torres**					

SOLUCIÓN Y RESULTADOS

I. Solución de Ejercicios / Problemas Propuestos

La presente práctica de laboratorio tiene como objetivo principal comprender, diseñar e implementar los algoritmos matemáticos y computacionales para la predicción y el cálculo balístico de trayectorias de proyectiles en entornos interactivos 2D y 3D utilizando el motor de videojuegos Unity. La práctica comprende dos grandes dimensiones de la física computacional aplicada:

- **Simulación Proyectiva Hacia Adelante (Forward Trajectory Simulation en 2D):** A partir de un impulso vectorialmente cuantificado aplicado por el jugador mediante la mecánica de arrastre táctil/ratón (slingshot), se proyectan de forma discreta las posiciones futuras de una bola física bajo el efecto constante de la aceleración de gravedad de la Tierra, representándolas mediante un haz visual de puntos graduados.
- **Cálculo Balístico Inverso y Apuntado Predictivo (Inverse Ballistic Trajectory en 3D):** Una Inteligencia Artificial montada sobre un tanque de combate calcula de manera analítica en tiempo real el ángulo exacto de elevación del cañón requerido para impactar a un objetivo móvil o estático a una distancia arbitraria, evaluando raíces cuadráticas para tiros tensos (Low Angle) y tiros curvos de obús/mortero (High Angle).



Figure 3.45: Parabolic arc

Figura 1. Modelo analítico de trayectoria parabólica bajo aceleración de gravedad constante. Fuente: Millington & Funge (2019).

1. Explicación del Funcionamiento de los Métodos de la Estructura de Clases del Videojuego 2D

El videojuego de predicción 2D se fundamenta en un patrón arquitectónico modular que separa estrictamente la física del proyectil, el cálculo y renderizado de la trayectoria, y el ciclo de vida del juego mediante el patrón Singleton. A continuación se desglosan las tres clases núcleo del sistema:

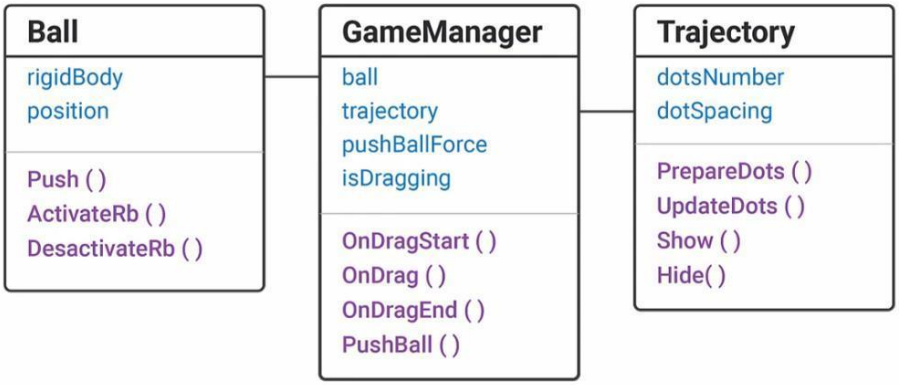


Figura 2. Diagrama de clases implementado para la predicción de trayectoria 2D. Fuente: Guía de Laboratorio 05.

Clase Ball (Ball.cs)

La clase Ball encapsula las propiedades cinemáticas y dinámicas del proyectil. Sus métodos y responsabilidades clave son:

- **Awake():** Obtiene y vincula en memoria las referencias locales a los componentes físicos 'Rigidbody2D' y 'CircleCollider2D' del GameObject al momento de su inicialización.
- **pos (Propiedad getter):** Expone de manera limpia y encapsulada la posición tridimensional 'transform.position' de la bola para que el trazador de trayectoria tome su origen exacto en cada fotograma.
- **Push(Vector2 force):** Aplica un impulso instantáneo a la bola mediante la función nativa del motor 'rb.AddForce(force, ForceMode2D.Impulse)'. ForceMode2D.Impulse resulta ideal para simular disparos o impactos, puesto que transfiere el cambio de momento de manera inmediata en un solo paso de simulación.
- **ActivateRb():** Establece 'rb.isKinematic = false', reintegrando la bola al control del motor de física NVIDIA PhysX/Box2D para que responda a la gravedad, fuerzas y colisiones.
- **DesactivateRb():** Detiene el movimiento de la bola anulando su velocidad lineal ('rb.linearVelocity = Vector3.zero') y su velocidad angular ('rb.angularVelocity = 0f'), y activa 'rb.isKinematic = true'. Esto congela el objeto en el espacio mientras el jugador se encuentra apuntando, evitando desplazamientos erráticos indeseados.

```
public class Ball : MonoBehaviour {
    [HideInInspector] public Rigidbody2D rb;
    [HideInInspector] public CircleCollider2D col;
    [HideInInspector] public Vector3 pos { get { return transform.position; } }

    void Awake () {
```

```
        rb = GetComponent<Rigidbody2D> ();
        col = GetComponent<CircleCollider2D> ();
    }

    public void Push (Vector2 force) {
        rb.AddForce (force, ForceMode2D.Impulse);
    }

    public void ActivateRb () { rb.isKinematic = false; }

    public void DesactivateRb () {
        rb.linearVelocity = Vector3.zero;
        rb.angularVelocity = 0f;
        rb.isKinematic = true;
    }
}
```

Clase Trajectory (Trajectory.cs)

Esta clase es la encargada de pronosticar y dibujar la curva parabólica en la pantalla. Funciona mediante un algoritmo discreto de muestreo temporal paso a paso:

- **PrepareDots():** Se ejecuta en el 'Start()' para instanciar previamente una lista fija de prefabs ('dotsNumber'). Los puntos se emparentan a un GameObject contenedor ('dotsParent') para mantener organizada la jerarquía. Además, calcula una atenuación de escala decreciente ('scaleFactor = dotMaxScale / dotsNumber'), de forma que los puntos iniciales sean de mayor tamaño y los finales más pequeños, aportando claridad visual sobre la dirección del proyectil.
- **UpdateDots(Vector3 ballPos, Vector2 forceApplied):** Calcula la posición predicha de cada marcador temporal. Emplea un paso de tiempo acumulativo 'timeStamp += dotSpacing'. Para cada punto i, resuelve analíticamente las ecuaciones de la cinemática clásica:

$$pos.x = ballPos.x + forceApplied.x \cdot timeStamp$$
$$pos.y = ballPos.y + forceApplied.y \cdot timeStamp - ½ \cdot |Physics2D.gravity| \cdot timeStamp^2$$

Nótese que la fuerza aplicada actúa directamente como vector de velocidad inicial en el modo 'Impulse' cuando la masa del proyectil es unitaria (m = 1 kg).

- **Show() y Hide():** Controlan la visibilidad de la trayectoria activando o desactivando en un solo llamado el contenedor raíz 'dotsParent.SetActive(bool)', garantizando un rendimiento óptimo al evitar activar o desactivar cada punto individualmente.

Clase GameManager (GameManager.cs)

Actúa como el orquestador central del juego y gestiona el flujo de interacción táctil o mediante ratón:

- **Patrón Singleton:** Garantiza una única instancia accesible globalmente ('GameManager.Instance') para que cualquier script ('Cup', 'ScoreManager', etc.) pueda notificar eventos sin acoplamientos rígidos.
- **OnDragStart():** Se activa al detectar 'Input.GetMouseButtonDown(0)'. Desactiva la física de la bola ('ball.DesactivateRb()'), captura la posición inicial del puntero en coordenadas del mundo

('cam.ScreenToWorldPoint(Input.mousePosition)') y visualiza los puntos guías llamando a 'trajectory.Show()'.

- **OnDrag():** Calcula continuamente el vector diferencial entre el punto inicial y la posición actual del ratón. Invierte el vector ('direction = (startPoint - endPoint).normalized') para emular la tensión de una resortera, multiplica por la distancia y por el escalar 'pushForce', y transmite la fuerza calculada a 'trajectory.UpdateDots(ball.pos, force)'.
- **OnDragEnd():** Se activa al soltar el botón ('Input.GetMouseButtonUp(0)'). Reactiva la física dinámica de la bola ('ball.ActivateRb()'), aplica el impulso 'ball.Push(force)' y oculta los puntos guía con 'trajectory.Hide()'.
- **Gestión de Reseteo y Seguridad:** Monitorea si la bola cae por debajo del umbral de pantalla ('ball.transform.position.y < outOfBoundsY = -7f') o si el jugador presiona manualmente la tecla [R], restableciendo la posición inicial sin romper la simulación.

2. Agregado de Otro Sprite (Cup) como Objetivo de Disparo

Para convertir la simulación en un juego interactivo con meta y desafío físico, se incorporó el sprite de una taza ('soap_cup') con un sistema de colisionadores híbrido que combina física sólida externa con detección volumétrica de entrada:

- **Física Sólida (EdgeCollider2D / BoxCollider2D):** Permite que la bola rebote de forma realista contra los bordes exteriores de la taza utilizando el material físico 'bounciness' con coeficiente de restitución e = 0.6.
- **Zona de Detección de Gol (BoxCollider2D Trigger):** Se ubicó un segundo colisionador configurado con 'isTrigger = true' estrictamente en la cavidad interior del recipiente (offset: (0, -0.35), tamaño: (0.9, 0.9)), de modo que únicamente active la anotación cuando la bola ingrese al interior.
- **Script Cup.cs:** Verifica mediante 'OnTriggerEnter2D' la entrada del proyectil, filtra colisiones espurias y notifica al 'GameManager'. Además, implementa 'OnDrawGizmosSelected()' para visualizar el área de anotación en color verde en la vista de escena de Unity, facilitando su calibración.

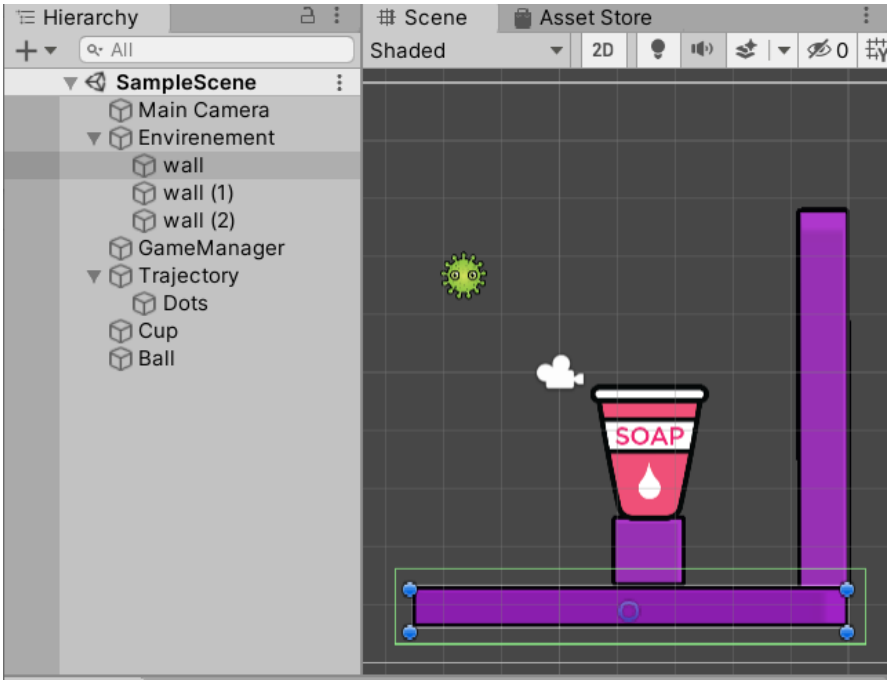


Figura 3. Configuración del GameObject Cup en el Inspector con EdgeCollider2D y trigger interno.

3. Conteo de Puntuación al Acertar en el Objetivo y Feedback Audiovisual

Se implementó una arquitectura completa de puntuación y feedback orientada a eventos mediante la clase 'ScoreManager.cs':

- **Control de Doble Registro (Anti-Double Scoring):** Mediante la bandera 'HasScoredCurrentShot' en 'GameManager', se garantiza que una bola que rebota varias veces dentro de la taza en un mismo tiro solo otorgue puntaje una única vez.
- **Persistencia del Récord (High Score):** El sistema almacena el puntaje máximo alcanzado entre sesiones mediante 'PlayerPrefs.SetInt("TrajectoryGame_HighScore", highScore)', promoviendo la rejugabilidad.
- **Efectos de Celebración:** Al ingresar la bola a la taza, 'Cup.cs' instancia el sistema de partículas 'Win_Particle System' con una ráfaga de confeti/estrellas y reproduce un sonido de victoria posicional mediante 'AudioSource.PlayClipAtPoint()'.
- **Interfaz Gráfica HUD en Tiempo Real (OnGUI):** Se diseñó un HUD ligero e interactivo mediante 'OnGUI()' que dibuja un panel oscuro en la esquina superior izquierda con sombreado de texto para SCORE y BEST, acompañado de un anuncio flotante emergente (+1 GOAL!) que realiza un desvanecimiento por interpolación lineal.
- **Reinicio Automático:** Tras encestar, 'GameManager' inicia una corrutina 'ResetBallAfterDelay(1.5f)' que espera a que termine la celebración antes de reposicionar la bola en el punto de lanzamiento.

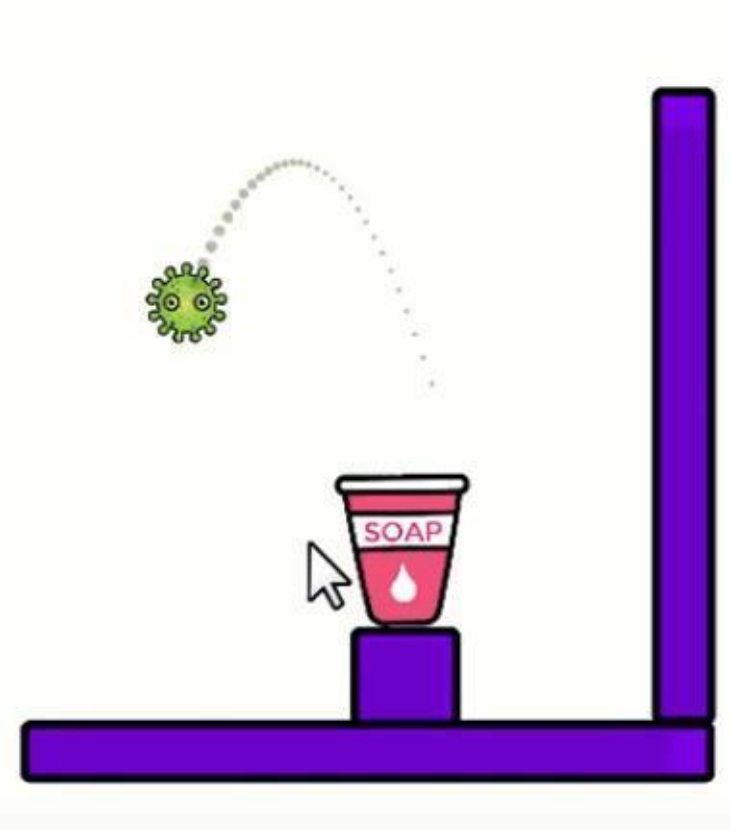


Figura 4. Ejecución del juego 2D: visualización de la trayectoria predictiva y objetivo Cup. Fuente: Proyecto desarrollado.

4. Desarrollo del Tutorial 'Calculating Trajectories - Unity Learn' (Balística 3D y Combate de Tanques)

El tutorial profundiza en el cálculo analítico inverso de trayectorias en un espacio tridimensional. En este escenario ('Tanks.unity'), el jugador conduce un tanque rojo ('Drive.cs') mientras un tanque enemigo gobernado por IA ('FireShell.cs') debe apuntar y dispararle proyectiles con precisión matemática sin importar la elevación ni la distancia relativa.

Deducción Matemática de la Ecuación Balística de Elevación

Dado un proyectil lanzado con rapidez inicial v desde el origen hacia un objetivo situado a distancia horizontal x y altura vertical y bajo gravedad g:

$$x = v \cdot \cos(\vartheta) \cdot t \implies t = x / (v \cdot \cos(\vartheta))$$
$$y = v \cdot \sin(\vartheta) \cdot t - ½ \cdot g \cdot t^2$$

Sustituyendo t en la segunda ecuación:

$$y = x \cdot \tan(\vartheta) - (g \cdot x^2) / (2 \cdot v^2 \cdot \cos^2(\vartheta))$$

Aplicando la identidad trigonométrica fundamental $1 / \cos^2(\theta) = 1 + \tan^2(\theta)$, obtenemos una ecuación cuadrática en términos de $\tan(\theta)$:

$$[(g \cdot x^2) / (2 \cdot v^2)] \cdot \tan^2(\vartheta) - x \cdot \tan(\vartheta) + [y + (g \cdot x^2) / (2 \cdot v^2)] = 0$$

Resolviendo mediante la fórmula cuadrática general para tan(θ):

$$\tan(\theta) = [v^2 \pm \sqrt{(v^4 - g \cdot (g \cdot x^2 + 2 \cdot y \cdot v^2))}] / (g \cdot x)$$

Análisis del Discriminante y Modos de Trayectoria

- **Condición de Alcance Balístico:** El discriminante $\Delta = v^4 - g \cdot (g \cdot x^2 + 2 \cdot y \cdot v^2)$ determina la viabilidad física del tiro. Si $\Delta < 0$, el objetivo se encuentra fuera del alcance balístico máximo para la velocidad v; en tal caso, 'CalculateAngle()' devuelve null, y el tanque de IA avanza automáticamente ('transform.Translate') para cerrar la distancia.
- **Ángulo Bajo (Low Trajectory Arc):** Corresponde a la raíz con signo negativo: $\tan(\theta) = (v^2 - \sqrt{\Delta}) / (g \cdot x)$. Produce una trayectoria rasante y directa con mínimo tiempo de vuelo, ideal para disparos rápidos a objetivos descubiertos.
- **Ángulo Alto (High Trajectory Arc):** Corresponde a la raíz con signo positivo: $\tan(\theta) = (v^2 + \sqrt{\Delta}) / (g \cdot x)$. Produce un tiro curvo parabólico muy elevado (estilo mortero u obús), con mayor tiempo de vuelo, capaz de impactar objetivos detrás de muros o desniveles pronunciados.

Implementación de los Scripts Balísticos 3D

- **FireShell.cs:** Calcula el ángulo llamando a 'CalculateAngle(useLowAngle)', rota la torreta ('turretBase.localEulerAngles = new Vector3(360f - angle, 0, 0)'), orienta el tanque hacia el jugador con 'Quaternion.Slerp' y dispara instanciando la bala con 'rb.linearVelocity = speed * turretBase.forward'.
- **AIShell.cs:** Monitorea la velocidad del proyectil en cada fotograma ('rb.linearVelocity') y alinea su orientación frontal mediante 'this.transform.forward = vel', logrando que la ojiva apunte en la dirección tangente a la trayectoria parabólica en todo instante.
- **Drive.cs:** Controla el tanque del jugador con los ejes horizontal y vertical, permite variar la inclinación manual del cañón con teclas [T] / [G] y disparar manualmente con la tecla [B].
- **TrajectoryHUD.cs:** Panel OnGUI superpuesto en el juego 3D que expone controles en caliente: muestra la distancia en tiempo real, alterna entre Ángulo Bajo y Alto con un botón, y ajusta mediante deslizadores la velocidad del proyectil (5 a 35 m/s) y la cadencia de fuego.

5. Resumen Comparativo de Técnicas de Trayectoria Implementadas

La combinación de los proyectos 2D y 3D permitió contrastar dos enfoques fundamentales de la simulación balística en videojuegos:

- **Naturaleza del Cálculo:** En 2D es una simulación discreta hacia adelante (Forward Steps) basada en la ecuación horaria $p(t) = p0 + v \cdot t + \frac{1}{2} \cdot g \cdot t^2$. En 3D es una resolución analítica inversa (Inverse Quadratic Ballistics) que resuelve directamente $\tan(\theta) = [v^2 \pm \sqrt{\Delta}] / (g \cdot x)$.
- **Entrada Principal:** En 2D la entrada es el vector de arrastre del puntero del ratón en la pantalla (Slingshot). En 3D la entrada es la posición espacial tridimensional del objetivo enemigo.

- **Soluciones Resultantes:** En 2D se obtiene una única parábola fija predicha. En 3D se obtienen dos posibles trayectorias válidas simultáneas (tiro rasante de ángulo bajo y tiro curvo de ángulo alto).
- **Respuesta Fuera de Rango:** En 2D los puntos caen por gravedad antes de la taza, indicando visualmente fuerza insuficiente. En 3D el discriminante se vuelve negativo ($\Delta < 0$), y la máquina de estados hace que el tanque avance automáticamente hasta quedar a tiro.

II. Solución del Cuestionario

1. ¿Cómo funciona el procedimiento de trayectoria de un proyectil?

El procedimiento de cálculo y simulación de la trayectoria de un proyectil se fundamenta en los principios de la mecánica clásica enunciados por Galileo Galilei e Isaac Newton. Este fenómeno físico obedece al Principio de Independencia de los Movimientos, según el cual el desplazamiento tridimensional puede descomponerse en ejes ortogonales independientes:

- **Movimiento Horizontal (Eje X y Eje Z):** En un entorno ideal sin resistencia de aire, no existen fuerzas netas actuando horizontalmente ($\Sigma Fx = 0$). Por la Primera Ley de Newton, la velocidad horizontal permanece constante ($vx = v0 \cdot \cos(\theta)$), describiendo un Movimiento Rectilíneo Uniforme (MRU): $x(t) = x0 + vx \cdot t$.
- **Movimiento Vertical (Eje Y):** Sobre el eje vertical actúa de forma continua la fuerza de atracción gravitatoria ($Fg = m \cdot g$ hacia abajo), produciendo una aceleración constante negativa $ay = -g$. La velocidad vertical disminuye linealmente ($vy(t) = vy0 - g \cdot t$), describiendo un Movimiento Rectilíneo Uniformemente Variado (MRUV): $y(t) = y0 + vy0 \cdot t - ½ \cdot g \cdot t^2$.
- **Parámetros Balísticos Notables:** A partir de estas relaciones se derivan de forma cerrada el tiempo total de vuelo hasta tocar el suelo ($tv = 2 \cdot v0 \cdot \sin(\theta)/g$), la altura máxima alcanzada ($Hmax = v0^2 \cdot \sin^2(\theta)/(2 \cdot g)$) y el alcance horizontal máximo ($Xmax = v0^2 \cdot \sin(2 \cdot \theta)/g$, cuyo valor máximo absoluto se da teóricamente a 45° en superficies planas).
- **Aproximaciones en Videojuegos:** A diferencia de la balística militar pura, en videojuegos interactivos frecuentemente se prescinde de la resistencia del aire por razones de determinismo y costo computacional, o se incrementa la gravedad (entre 1.5× y 3× del valor terrestre de 9.81 m/s^2) para que los saltos y proyectiles se sientan rápidos y reactivos en lugar de 'flotantes'.

2. ¿Qué elementos proporciona Unity para apoyar la implementación de la trayectoria de proyectiles en videojuegos?

Unity provee un ecosistema integral de componentes, motores físicos y APIs matemáticas que simplifican el diseño, renderizado y predicción de proyectiles:

- **Motores de Física Dedicados (PhysX y Box2D):** Unity integra el motor NVIDIA PhysX para simulaciones tridimensionales y Box2D para entornos bidimensionales. Los componentes 'Rigidbody' y 'Rigidbody2D'

gestionan la integración numérica continua de aceleraciones, velocidades, masa, inercia y arrastre (linearDrag).

- **Modos de Aplicación de Fuerza (ForceMode):** La API 'AddForce()' permite aplicar fuerzas según la necesidad física: 'ForceMode.Impulse' (o ForceMode2D.Impulse) para detonaciones o disparos instantáneos donde se modifica el momento en un único frame; 'ForceMode.Force' para propulsiones continuas de cohetes; y 'ForceMode.VelocityChange' para alterar la velocidad ignorando la masa.
- **Detección Continua de Colisiones (Continuous Collision Detection - CCD):** Los proyectiles a alta velocidad sufren frecuentemente del problema de tunelización ('tunneling'), atravesando colisionadores delgados entre dos frames de física consecutivos. Unity soluciona esto con 'CollisionDetectionMode.Continuous' y 'ContinuousDynamic', barriendo el volumen geométrico del proyectil a lo largo de su vector de desplazamiento.
- **Simulación Física Secundaria y Predictiva (Physics.Simulate):** A partir de versiones recientes, Unity permite crear escenas de física aisladas ('PhysicsScene') y ejecutar 'Physics.Simulate(deltaTime)' de manera manual y controlada. Esto hace posible proyectar trayectorias exactas que calculan colisiones y múltiples rebotes en tiempo real antes de efectuar el disparo real.
- **Herramientas de Renderizado de Trazas:** 'LineRenderer' y 'TrailRenderer' permiten dibujar trayectorias suaves en el espacio 3D mediante spline o arreglos de vértices, mientras que los sistemas de partículas ('VFX Graph' y 'Shuriken') generan estelas de humo, fuego o chispas a lo largo de la parábola.
- **Consultas Espaciales Geométricas (Physics.Raycast / CircleCast / SphereCast):** Permiten realizar barridos volumétricos predictivos para anticipar el punto exacto y la normal de impacto antes de que el proyectil alcance físicamente el obstáculo.

III. Conclusiones

- **Equivalencia entre Modelos Analíticos y Simulación Discreta:** Se demostró que la formulación cinemática continua de proyectiles puede discretizarse con alta fidelidad gráfica en Unity mediante pasos de tiempo finitos (Δt = dotSpacing), permitiendo al jugador visualizar con exactitud milimétrica el punto de impacto antes de ejecutar el lanzamiento.
- **Versatilidad de la Ecuación Balística Cuadrática en IA:** La resolución analítica de $\tan(\theta)$ para el combate de tanques en 3D proporciona dos soluciones válidas (tiro directo de ángulo bajo y tiro curvo de ángulo alto). Esto permite diseñar comportamientos tácticos inteligentes donde la IA elige el tiro tenso por velocidad o el tiro parabólico para sortear obstáculos del terreno.
- **Gestión de Alcance y Transición de Estados:** El análisis en tiempo real del discriminante balístico ($\Delta < 0$) dota a los agentes autónomos de conciencia espacial sobre su alcance efectivo, posibilitando una transición fluida entre estados de maniobra (perseguir y avanzar) y estados de combate (apuntar torreta y abrir fuego).

- **Arquitectura Modular y Desacoplada:** La integración del patrón Singleton, colisionadores de disparo tipo trigger, eventos de C# y componentes dedicados ('Ball', 'Cup', 'ScoreManager', 'FireShell') permitió construir una base de código robusta, escalable y mantenible con persistencia de datos (PlayerPrefs) y feedback audiovisual completo.
- **Compatibilidad y Evolución Tecnológica del Motor:** El desarrollo evidenció la importancia de adaptarse a las actualizaciones de la API de Unity (como la transición hacia 'rb.linearVelocity' en Unity 6 mediante directivas de compilación condicional '#if UNITY_6000_0_OR_NEWER'), asegurando la longevidad y portabilidad multiplataforma del proyecto.

RETROALIMENTACIÓN GENERAL

La práctica de laboratorio integró satisfactoriamente los fundamentos matemáticos de la cinemática clásica con las herramientas de simulación física en tiempo real de Unity. El ejercicio 2D demostró el poder visual de la predicción paso a paso para mecánicas de tipo puzzle o artillería informal (Angry Birds, canastas físicas), mientras que el ejercicio 3D de tanques evidenció la importancia del álgebra y la trigonometría para agentes autónomos que requieren precisión balística sin depender de tanteos empíricos.

Como elemento de mejora e investigación futura, se recomienda explorar la incorporación de resistencia de aire dinámica (Drag cuadrático dependiente de la densidad atmosférica) e integración de rebotes analíticos continuos mediante 'PhysicsScene.Simulate' en una escena secundaria paralela, lo que permitiría predecir trayectorias complejas con colisiones elásticas múltiples sobre geometrías irregulares.

REFERENCIAS Y BIBLIOGRAFÍA

[1] I. Millington and J. Funge, Artificial Intelligence for Games, 3rd ed. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group, 2019, pp. 120–128.

[2] Unity Technologies, 'Calculating Trajectories - Unity Learn Tutorial', Unity Technologies, 2024. [Online]. Disponible: https://learn.unity.com/tutorial/calculating-trajectories

[3] Unity Technologies, 'Rigidbody2D API Reference and Simulation Modes', Unity Documentation, 2024. [Online]. Disponible: https://docs.unity3d.com/ScriptReference/Rigidbody2D.html

[4] Unity Technologies, 'Physics2D.gravity and Multi-Scene Physics Simulation', Unity Manual, 2024. [Online]. Disponible: https://docs.unity3d.com/Manual/MultiScenePhysics.html

[5] J. Gregory, Game Engine Architecture, 3rd ed. Boca Raton, FL, USA: CRC Press, 2018, pp. 785–830.

[6] D. H. Eberly, Game Physics, 2nd ed. San Francisco, CA, USA: Morgan Kaufmann Publishers, 2010.